Tollgate

System Architecture

Bot-Economics Payment Infrastructure on Arc

Cloudflare for AI bots that pays publishers instead of blocking them.

Every AI scrape becomes a transaction, not a theft.

This document specifies every service, contract, data table, API endpoint, sequence, and integration needed to monetize bot traffic through sub-cent USDC payments settled on Arc.

3 Products **15 Integrations** **11 Tables** **23 Endpoints** **1 Chain**

Version: 1.0 **Date:** April 2026 **Classification:** Hackathon Submission / Public

Author: Brian Mwai **Built for:** Agentic Economy on Arc – Circle Hackathon, SF, 2026

github.com/brn-mwai/tollgate ◇ tollgate.brianmwai.com

Contents

Preface

What this document is

This is the **complete system architecture** for Tollgate, a payment rail that lets web publishers charge AI bots a fraction of a cent per request in USDC, settled on Arc with sub-second finality.
It follows a seven-step design framework: **Clarify** requirements, **Estimate** capacity, **Architect** the high-level system, **Detail** workflows, **Design** APIs and schema, **Scale** along a growth path, and **Verify** that non-functional requirements are met by concrete design decisions.
Every box, table, and diagram in this document is justified by a requirement. Every external call is isolated in an action. Every query has an index. This is a build specification, not a pitch deck.

The single sentence we repeat

Tollgate turns every AI scraper request from a theft into a transaction.

1 Clarify: Requirements

Context answered first

User roles. Publisher (website owner), Agent (AI bot operator), Admin (Tollgate staff).
DAU targets. Launch: 100 publishers $\times$ 50 agents $\approx$ 5K DAU. Year 1: 10K publishers $\times$ 1K agents $\approx$ 500K DAU. Year 3: 100K publishers $\times$ 50K agents $\approx$ 5M DAU.
Workload profile. Heavily write-heavy. Each request produces one to three database writes. Read-to-write ratio sits near 1:20, which pushes the design toward materialized rollups rather than live aggregation.
Regulated data. No PII on agents (wallet address is the only identifier). Publishers are KYCd through Circle Wallets. Every onchain transaction is public by design. Tollgate never touches money transmission: the publisher is the merchant of record, Tollgate is the SaaS that sits between.

1.1 Functional Requirements

ID	Requirement
F-01	Publisher can sign up, verify site ownership, install middleware via SDK or hosted Worker.
F-02	Publisher configures per-path pricing, bot whitelists, blacklists, and reputation tiers.
F-03	Publisher receives a programmable Circle Wallet on Arc at onboarding. No key management required.
F-04	Middleware returns HTTP 402 for unpaid bot requests, with price, nonce, recipient, chain, and expiry.
F-05	Agent SDK auto-detects 402, signs USDC transfer on Arc, retries request with payment proof.
F-06	Middleware verifies proof via x402 facilitator and serves content on success.
F-07	Middleware issues short-TTL HMAC receipts so repeat requests skip onchain round-trip.
F-08	Control plane aggregates events in real time and streams live earnings to the publisher dashboard.
F-09	Publisher can withdraw USDC to any chain via Circle CCTP and off-ramp to bank.
F-10	Agents accrue onchain reputation via ERC-8004 contract, feeding tiered discount pricing.
F-11	Admin can suspend abusive wallets, moderate disputes, and view network-wide health.
F-12	Agents can query AIsa Nanopayment APIs through the same SDK flow for upstream data needs.

1.2 Non-Functional Requirements

NFR	Target	Rationale
Availability (middleware)	99.99%	Publisher sites must not go down because of Tollgate.
Availability (control plane)	99.95%	Dashboard outage tolerable; payments must keep flowing.
p95 latency (cached request)	$<$ 50 ms	Receipt path must feel invisible to scrapers.
p95 latency (cold request)	$<$ 1.2 s	First payment: 402 round-trip + Arc sign + facilitator verify.
Consistency (wallet balance)	Strong	Circle Wallets as source of truth; reconcile nightly.
Consistency (analytics)	Eventual	Batched 5s ingest; hourly rollups; publisher accepts lag.
Throughput target (Y3)	10K req/s sustained, 50K peak	Network-effect driven scale curve.
Compliance	SOC 2 Type II by Y1	Enterprise publisher sales gate.
Privacy	GDPR / CCPA for publishers	Agents are pseudonymous by protocol.
Security	Nonce replay, signature, HMAC	Standard for payment systems + x402 spec.

1.3 Out of Scope

- Human micropayments. Bots only. Every prior micropayment startup died trying to charge humans.
- Fiat rails (Stripe, ACH) for payment-in. USDC only at MVP. Off-ramp is fiat-aware via Circle.

- Non-Arc chains at launch. Bridge Kit adds support post-MVP without changing the SDK surface.
- Content-level DRM. We gate access; we do not police downstream reuse.
- Human CAPTCHA. Publishers pass human traffic through before Tollgate sees it.
- Bot classification. We trust the client identifier; misuse is priced out by reputation over time.

1.4 Assumptions

- Publisher already runs a web server capable of hosting middleware (Express, Hono, CF Worker, Next.js, nginx sidecar).
- Agent operator can sign transactions programmatically, either through a self-custody Arc wallet or through Circle Wallets API.
- Arc testnet and mainnet are reachable via RPC. Official faucet funds demo wallets.
- The x402 facilitator (Coinbase-hosted at MVP, self-hosted at scale) returns verdicts in under 400 ms.
- Circle Wallets API provides adequate throughput for per-publisher wallet provisioning.
- AIsa APIs expose Nanopayment-ready endpoints that Tollgate agents can call natively.

2 Estimate: Capacity

Sizing the system

Base unit of work is the **event row**: one HTTP request to a protected path produces one event. Every other estimate derives from this.

2.1 Year 1 Capacity Table

Metric	Value	Formula / Notes
Daily events	100M / day	500K DAU $\times$ 200 req / user / day avg
Monthly events	3B / month	Daily $\times$ 30
Avg row size (events)	320 B	Indexed fields + optional metadata
Storage, hot events (90d, 1.3 replication)	3.7 TB	320 B $\times$ 100M $\times$ 90 $\times$ 1.3
Ingress + egress bandwidth	200 GB / day	2 KB $\times$ 100M
Peak QPS	3.5K QPS	$(100\text{M} \times 3)/86400$
Concurrent connections	35K	DAU $\times 0.07$
Convex function calls / month	$\sim$ 6.2B	3B events $\times$ 2 writes + 200M reads
Onchain TX uncached (ceiling)	$\sim$ 900M / month	30% miss rate without receipts
Onchain TX with receipts (target)	$\sim$ 30M / month	50x collapse via 5-min HMAC receipts
Clerk MAU (publisher accounts)	$\sim$ 10K	Publisher seats only

2.2 Read to Write Analysis

Design implication

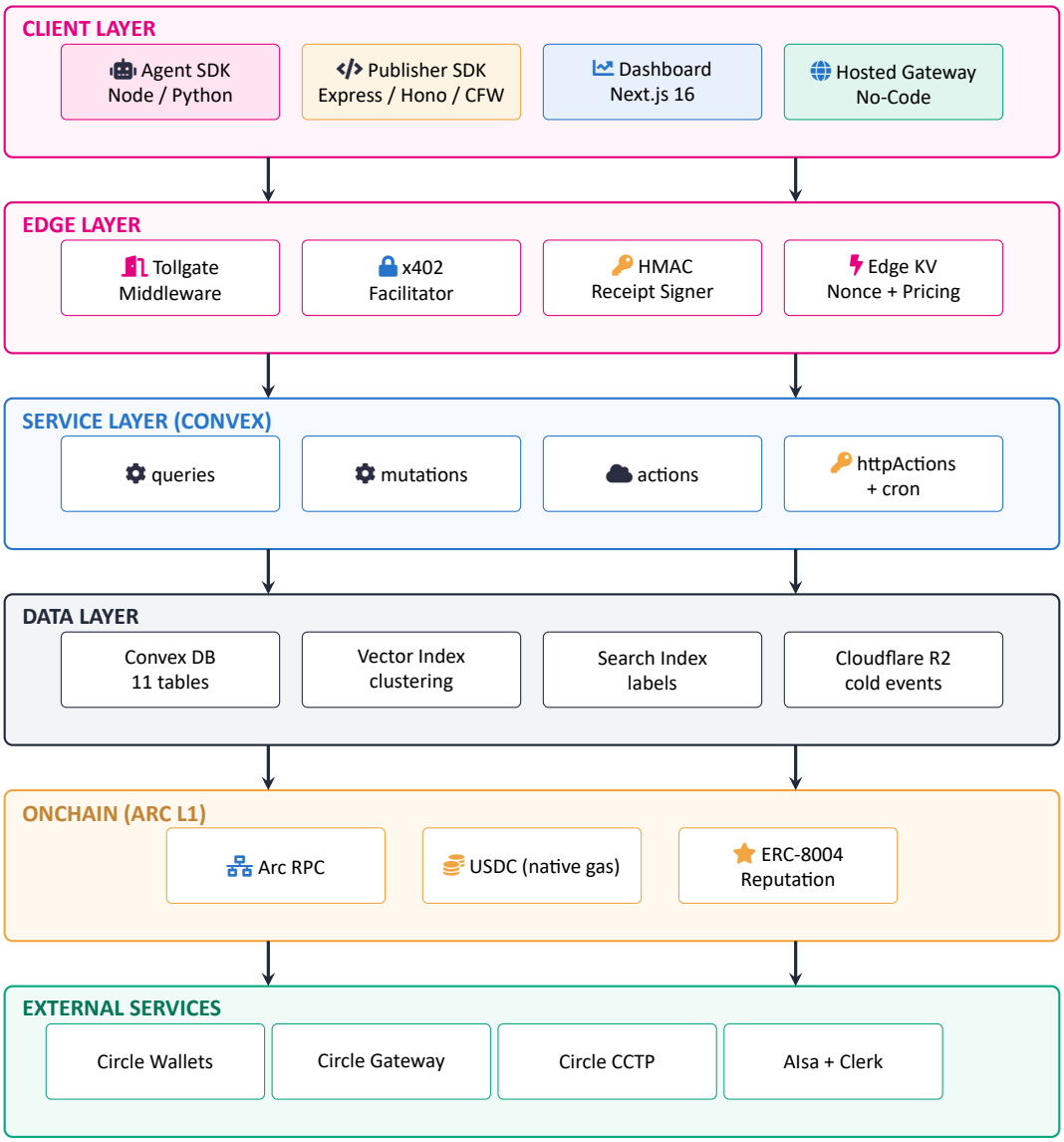
With 6.2 billion Convex function calls per month at Year 1, running the dashboard off raw events becomes an economic hazard. The design therefore relies on:

- Edge-side batching of telemetry (5-second flush window, 100-event cap per batch).
- Hourly rollup tables that the dashboard reads from, not raw events.
- Seven-day hot retention in Convex, older events archived to Cloudflare R2.
- Receipt caching at the middleware layer, collapsing 50 requests into 1 onchain TX.

Without this stack of compressions, Convex billing alone becomes the dominant cost and the product becomes un-shippable. With it, control-plane cost stays well inside a healthy gross margin even at Year 1 scale.

3 Architect: High-Level Design

3.1 Six-Layer Architecture



Reading this diagram. Top to bottom, data and trust flow downward: clients issue requests, the edge handles gating and payment verification, Convex holds application state, data layer persists, onchain settles, external services provide value. Each band can scale independently because responsibilities are cleanly separated.

3.2 Component Responsibilities

Component	Responsibility & why chosen	Rejected alternative
Middleware SDK	Runs in publisher process for lowest latency. One-line install.	DNS-level proxy: adds hop, violates permissionless positioning.
Hosted Gateway	Managed Cloudflare Worker for non-technical publishers (WordPress, Ghost).	Plugin-only: excludes long-tail, caps growth.
Convex Functions	Reactive reads, transactional writes, vector, cron, file storage in one platform.	Postgres + Redis + Kafka: three systems, no reactivity, higher ops burden.
x402 Facilitator	Onchain proof verification + nonce dedup + caching. Coinbase-hosted at MVP.	DIY verify: no dedup, no cache, duplicated effort across deployments.
Arc L1	Settlement layer. USDC-denominated gas gives deterministic margin math.	Ethereum / Polygon / Base: volatile gas destroys sub-cent unit economics.
Circle Wallets	Publisher custody abstraction. Zero-key onboarding.	Self-custody wallet: kills publisher conversion at signup.
Circle Gateway	Agent-side unified USDC across chains. One integration, any chain.	Per-chain wallet config: agent operators refuse integration complexity.
Circle Bridge Kit	Publisher off-ramp to Base / Ethereum then to bank.	Manual bridging: dashboard UX breaks.
ERC-8004 (Vyper)	Onchain agent reputation. Portable across networks.	Off-chain reputation DB: centralized, not portable, not agent-friendly.
Clerk	Publisher auth, JWT template for Convex.	Custom auth: compliance burden, no SOC 2 inheritance.
Cloudflare R2	Cold event archive beyond seven days.	Convex storage: not priced for multi-TB cold data.
AIsa APIs	Upstream data source consumed through same Nanopayment flow. Dogfood.	Skip: loses ”Tollgate as both provider and consumer” narrative.

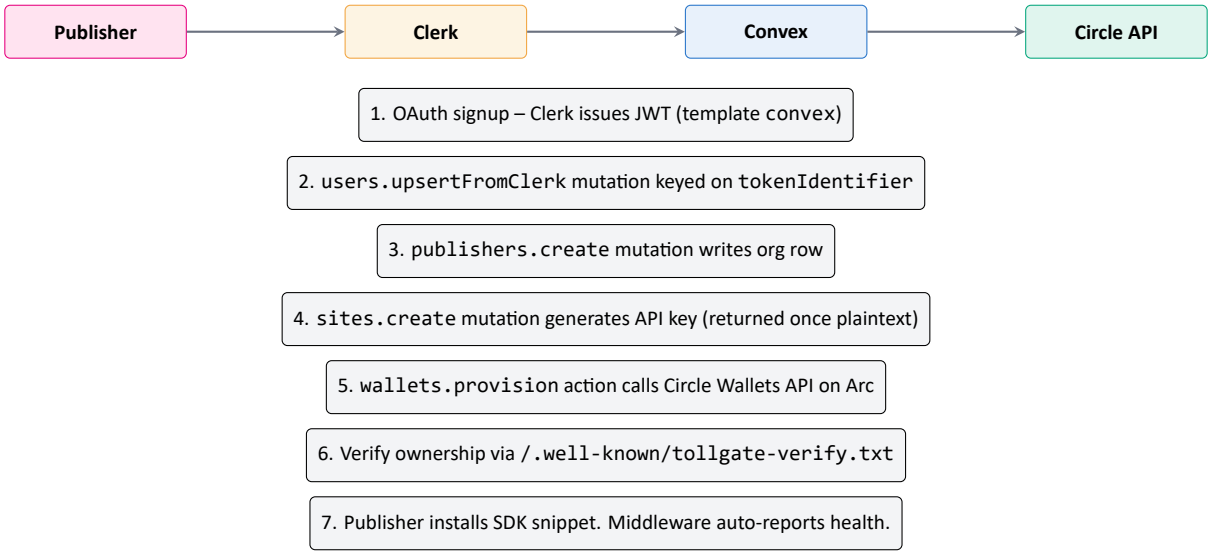
4 Detail: Critical Workflows

Every workflow below traces end-to-end: trigger, client call, auth check, database mutation, external call, response shape, failure mode.

4.1 Workflow A: Publisher Onboarding

Path A — From Clerk signup to live middleware

Trigger: Publisher visits `tollgate.brianmwai.com` and clicks sign up.
Outcome: Publisher has a funded Circle Wallet on Arc, an active site with API key, and a one-line install snippet.

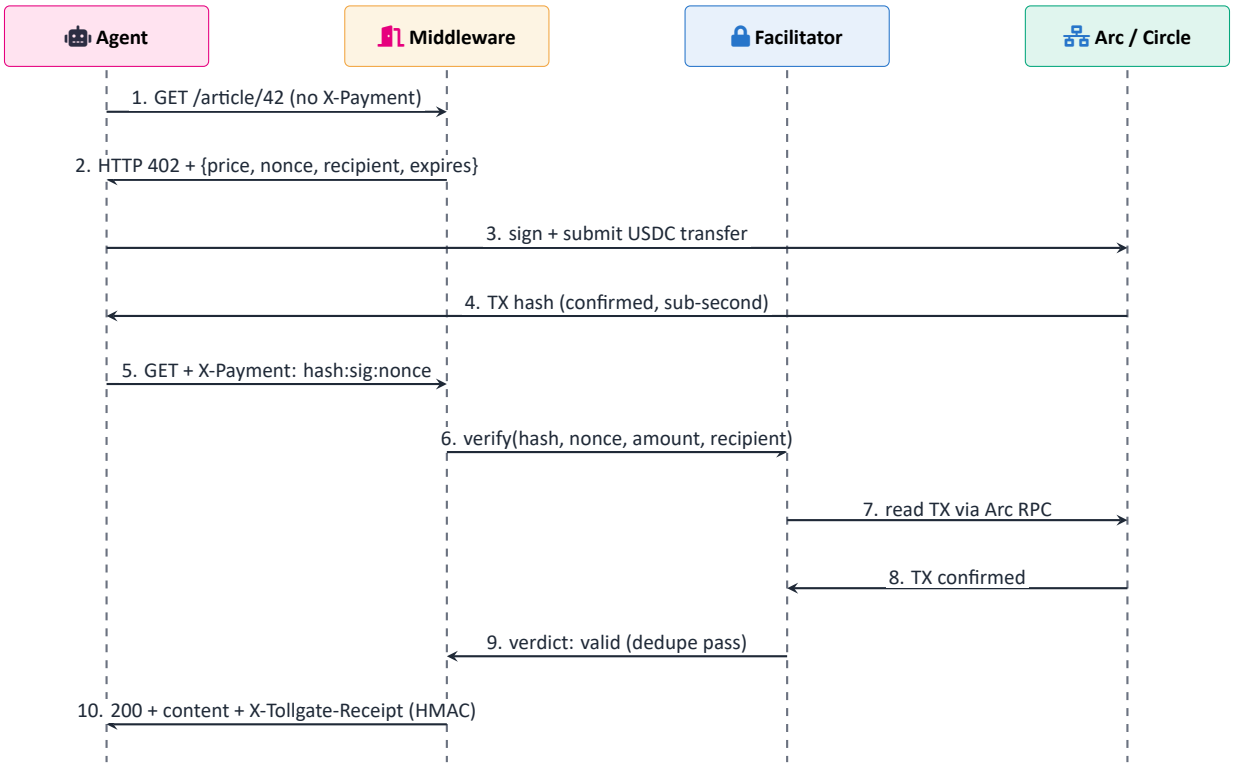


Failure modes. Circle API timeout: retry three times with exponential backoff, queue via `ctx.scheduler.runAfter`. Ownership file missing: site stays `unverified`, 402 disabled, publisher prompted with exact remediation.

4.2 Workflow B: Paid Agent Request (Cold, Uncached)

Path B — 402 to 200 in under one second

Trigger: AI agent issues GET to a protected path with no payment header.
Outcome: One onchain USDC TX, one content delivery, one event recorded, one receipt issued.



Failure modes. Facilitator timeout beyond five seconds: fall through to fail-open if publisher opts in, otherwise another 402. Signature mismatch: reject with fresh nonce. Double-spend: rejected by Arc, returns 402.

4.3 Workflow C: Paid Agent Request (Warm, Cached Receipt)

Path C — sub-50ms served request

Trigger: Agent sends GET with a valid X-Tollgate-Receipt header.
Outcome: Middleware HMAC-verifies locally. No Convex call. No facilitator call. No onchain TX. Content served.

This collapses 50-request scrape sessions into a single onchain TX plus 49 cached hits, reducing gas spend fiftyfold at the scraper level while still allowing the publisher to settle in real time on the first request.

4.4 Workflow D: Publisher Withdraw

1. Publisher clicks Withdraw $45.23 in dashboard
2. `withdrawals.request` mutation: validate balance, create pending row
3. `withdrawals.execute` action: Circle Wallets transfer API; optionally CCTP bridge
4. Webhook from Circle confirms TX; `http.circleWebhook` verifies HMAC
5. Update row to `sent`; emit notification event

Failure modes. TX reverted: mark `failed`, refund virtual balance. Circle webhook missing: poll via scheduled function every 15 s for up to one hour.

4.5 Workflow E: Reputation Roll-Up (Daily)

1. `cron.reputation.roll` runs daily at 00:00 UTC
2. Query rolled events per agent wallet for last 24 h
3. Compute score: payment rate, dispute count, volume, consistency
4. Batch-push score updates to ERC-8004 registry on Arc
5. Flag agents above threshold for tiered discount pricing

5 Design: API and Schema

5.1 Convex Schema

```
// convex/schema.ts -- 11 tables, every query backed by an index
import { defineSchema, defineTable } from "convex/server"
import { v } from "convex/values"

export default defineSchema({
  users: defineTable({
    tokenIdentifier: v.string(),
    email: v.string(),
    name: v.optional(v.string()),
    role: v.union(v.literal("publisher"), v.literal("admin")),
  }).index("by_token", ["tokenIdentifier"])
    .index("by_email", ["email"]),

  publishers: defineTable({
    ownerId: v.id("users"),
    orgSlug: v.string(),
    plan: v.union(v.literal("free"), v.literal("pro"), v.literal("
   enterprise")),
    circleWalletId: v.string(),
    arcAddress: v.string(),
    balanceUsdc: v.string(),       // string for on-chain precision
    platformFeeBps: v.number(),
  }).index("by_owner", ["ownerId"])
    .index("by_slug", ["orgSlug"])
    .index("by_address", ["arcAddress"]),

  sites: defineTable({
    publisherId: v.id("publishers"),
    domain: v.string(),
    apiKeyHash: v.string(),
    status: v.union(v.literal("unverified"),
                    v.literal("active"),
                    v.literal("suspended")),
    verifyToken: v.string(),
    failOpenOnFacilitator: v.boolean(),
  }).index("by_publisher", ["publisherId"])
    .index("by_domain", ["domain"])
    .index("by_keyhash", ["apiKeyHash"]),

  pricingRules: defineTable({
    siteId: v.id("sites"),
    pathPattern: v.string(),
    priceMicroUsdc: v.number(),
    botClass: v.optional(v.string()),
    priority: v.number(),
  }).index("by_site_priority", ["siteId", "priority"]),

  agents: defineTable({
    walletAddress: v.string(),
    label: v.optional(v.string()),
    reputationScore: v.number(),
    totalPaidMicroUsdc: v.string(),
    firstSeenAt: v.number(),
    status: v.union(v.literal("active"),
                    v.literal("flagged"),
```

```
                  v.literal("banned")),
}).index("by_wallet", ["walletAddress"])
  .index("by_score", ["reputationScore"])
  .searchIndex("search_label", { searchField: "label" }),

events: defineTable({
  siteId: v.id("sites"),
  agentWallet: v.string(),
  path: v.string(),
  status: v.union(
    v.literal("paid_onchain"),
    v.literal("paid_cached"),
    v.literal("unpaid_402"),
    v.literal("rejected"),
    v.literal("failed_verify")
  ),
  priceMicroUsdc: v.number(),
  txHash: v.optional(v.string()),
  nonce: v.string(),
  occurredAt: v.number(),
}).index("by_site_time", ["siteId", "occurredAt"])
  .index("by_agent_time", ["agentWallet", "occurredAt"])
  .index("by_txhash", ["txHash"])
  .index("by_nonce", ["nonce"]),

hourlyRollup: defineTable({
  siteId: v.id("sites"),
  hourBucket: v.number(),
  totalRequests: v.number(),
  paidRequests: v.number(),
  earnedMicroUsdc: v.number(),
  uniqueAgents: v.number(),
}).index("by_site_hour", ["siteId", "hourBucket"]),

withdrawals: defineTable({
  publisherId: v.id("publishers"),
  amountMicroUsdc: v.string(),
  destination: v.string(),
  status: v.union(v.literal("pending"),
                  v.literal("sent"),
                  v.literal("failed")),
  circleTxId: v.optional(v.string()),
  requestedAt: v.number(),
}).index("by_publisher_time", ["publisherId", "requestedAt"]),

receipts: defineTable({
  hmacKey: v.string(),
  siteId: v.id("sites"),
  rotatedAt: v.number(),
}).index("by_site", ["siteId"]),

nonceLog: defineTable({
  nonce: v.string(),
  siteId: v.id("sites"),
  seenAt: v.number(),
}).index("by_nonce", ["nonce"]),

auditLog: defineTable({
  actor: v.string(),
  action: v.string(),
```

```
    entity: v.string(),
    entityId: v.string(),
    meta: v.any(),
    occurredAt: v.number(),
  }).index("by_actor_time", ["actor", "occurredAt"])
    .index("by_entity_time", ["entity", "entityId", "occurredAt"]),
})
```

5.2 Function Surface

Function	Type	Auth	Purpose
`users.upsertFromClerk`	mutation	Clerk	Upsert identity keyed on `tokenIdentifier`.
`publishers.create`	mutation	user	Create publisher org row.
`publishers.get`	query	user	Return current user's publisher record.
`sites.create`	mutation	publisher	Create site, generate API key, return plaintext once.
`sites.list`	query	publisher	Paginated site list.
`sites.verify`	action	publisher	Fetch `/.well-known` and confirm ownership.
`sites.rotateKey`	mutation	publisher	Rotate API key, log action.
`pricingRules.upsert`	mutation	publisher	Add or update price per path or bot class.
`pricingRules.list`	query	publisher	Return rules for a site.
`wallets.provision`	action	publisher	Call Circle Wallets API, store wallet.
`wallets.balance`	action	publisher	Read Arc + Circle balances.
`withdrawals.request`	mutation	publisher	Create pending withdrawal row.
`withdrawals.execute`	action	internal	Execute Circle transfer; optionally CCTP bridge.
`events.ingestBatch`	internalMutation	gateway	Batch-insert from edge.
`events.listForSite`	query	publisher	Paginated event log.
`hourlyRollup.forSite`	query	publisher	Dashboard time-series.
`agents.lookup`	query	public	Price lookup for middleware.
`reputation.pushOnchain`	action	internal	Batch-write ERC-8004 scores.
`cron.reputation.roll`	scheduled	system	Daily reputation update.
`cron.rollupHourly`	scheduled	system	Hourly aggregation.
`cron.cleanupNonces`	scheduled	system	Trim nonceLog every 10 minutes.
`cron.archiveColdEvents`	scheduled	system	Daily archive to R2.
`http.circleWebhook`	httpAction	HMAC	Receive Circle callbacks.
`http.telemetryIngest`	httpAction	API key	Edge batches events via this endpoint.

5.3 Client APIs

Endpoint	Method	Purpose
`/.well-known/tollgate.json`	GET	Publisher site advertises pricing config.
`/v1/verify`	POST	Facilitator verifies payment proof.
`/v1/telemetry`	POST	Edge gateway ingests event batches.
`/v1/pricing`	GET	Agent-class lookup for dynamic pricing.
`/v1/receipts/rotate`	POST	Admin rotation of HMAC keys.
`/webhooks/circle`	POST	Circle callbacks (HMAC-signed).
`/webhooks/clerk`	POST	Clerk lifecycle webhooks.

6 Scale: Growth Path

Four stages from 0 to 1M DAU

Each stage below is a concrete infra move, not a vague ”scale up”. The trigger for moving up a stage is a combination of DAU, monthly Convex bill, and p95 latency drift.

Stage	DAU	Convex	Edge + Data	Onchain	Auth + FE
1	0-5K	Default plan, raw events 7d retention	Single CF Worker gateway	Coinbase-hosted facilitator	Clerk Free, Vercel Hobby
2	5K-100K	Pro plan, 5s batch telemetry, rollup aggregates	R2 for archive beyond 7d	Self-host facilitator in one region	Clerk Pro + SSO, Vercel Pro
3	100K-1M	Split hot (24h) / warm (7d) / cold (R2); vector index for agent clustering; Convex components per cohort	CF Workers in 5 regions; Durable Objects for nonce dedup	Multi-region facilitator; dedicated Arc RPC tier	Clerk Enterprise, Vercel multi-region edge
4	1M+	Shard by publisher cohort via Convex components; optional Kafka ingest feeding Convex every 10s	ClickHouse warehouse for cold analytics API	Run own Arc validator; self-host x402 HA cluster; own ERC-8004 operator	Clerk Enterprise + custom claims, dedicated edge infra

6.1 Convex-Specific Levers

- Every list uses `paginate(opts)`; no unbounded `.collect()` anywhere.
- Every query is backed by an index. Any new query requires an index migration.
- `internalMutation` and `internalAction` isolate trust boundaries from client-callable surfaces.
- `ctx.scheduler.runAfter` handles retries and deferred work without external queue infra.
- `withSearchIndex` powers agent label search from the admin console.
- `withVectorIndex` enables behavior clustering for anomaly detection at Stage 3.
- Convex components isolate per-cohort state at Stage 4 when single-deployment tenancy strains.

7 Verify: NFRs Mapped to Decisions

NFR	Design decision that satisfies it
Middleware p95 $<$ 50 ms cached	HMAC receipts verified in-process; no Convex or facilitator call on warm path.
Middleware p95 $<$ 1.2 s cold	Facilitator same region as publisher; Arc sub-second finality; pricing rules edge-cached.
99.99% middleware uptime	Middleware lives in publisher process; Gateway calls are async; Tollgate outage never blocks publisher.
99.95% control-plane uptime	Convex Cloud SLA + Vercel multi-region; dashboard down does not stop payments.
Strong balance consistency	Convex ACID mutations; Circle Wallets source of truth; nightly reconcile.
Eventual analytics consistency	5s batched ingest; hourly rollups; explicit lag disclosed to publisher.
10K sustained QPS at Y3	Stateless edge middleware, horizontal scale; Gateway batching; rollups reduce Convex writes 20x.
Nonce replay prevention	Per-site LRU at middleware (1 min TTL) + Convex `nonceLog` index + facilitator-side dedup.
Signature verification	x402 spec + ECDSA at facilitator + HMAC on receipts, three-layer defense.
Secrets isolation	API keys SHA-256 hashed; plaintext returned once; Circle Wallets custodies signing keys; Convex env vars for HMAC.
SOC 2 Type II by Y1	Audit log table + R2 archive; Clerk, Convex, Vercel all SOC 2 attested; Vanta / Drata during Y1.
GDPR / CCPA for publishers	Clerk DPA signed; pseudonymous agents by design; publisher deletion flow.
Permissionless agent access	No signup required; wallet address is identity; no KYC for agents.

7.1 Gaps and Mitigations

Known gaps at MVP

Facilitator single-point-of-failure. Coinbase-hosted at launch. Self-host at Stage 2. Contingency: `failOpenOnFacilitator` flag lets publishers serve content if verify is unreachable beyond five seconds.

Convex cost at 1M+ DAU. Projected function-call volume will challenge Pro plan economics. Stage 4 moves raw ingest to Kafka + ClickHouse. Trigger threshold: monthly Convex bill exceeds $50K.

Arc chain risk. Arc is a new network. Mitigation: Circle Bridge Kit ready to route via Base or Solana without SDK contract change. Monitor Arc uptime alongside our own.

Wallet key custody. Circle Wallets is custodial for publishers at MVP. Pro-plan users can opt into self-custody smart wallets from Stage 2.

Bot detection. Tollgate does not classify bots vs humans. Responsibility sits with publisher (User-Agent, Cloudflare Bot Management, WAF). This is explicit in the SDK docs.

8 Resource Integration Map

Every sponsor tool used, justified

Tollgate uses all 15 Circle, Arc, Vyper, and AIsa resources offered by the hackathon. Each is a judge-signal and each strengthens the product.

Resource	Tier	Role in Tollgate
Arc L1	Required	Settlement layer. Every payment TX lands on Arc.
USDC	Required	Payment + gas token. Sub-cent denominated.
Circle Nanopayments	Required	Core billing primitive. Every 402 resolves to a Nanopayment.
Circle Wallets	Core	Publisher wallets auto-provisioned. Custodial by default.
Circle Gateway	Differentiator	Agent-side unified USDC balance across chains.
Circle Bridge Kit (CCTP)	Polish	Publisher off-ramp: Arc to Base to Ethereum to bank.
x402 facilitator	Core protocol	Payment proof verification + nonce dedup.
Circle Developer Console	Submission artifact	Required in video; shows TX executed and verified.
Testnet faucet	Required	Fund four demo wallets: publisher + three agents.
Circle Developer Blog	Trust signal	Reference best-practice patterns. Cite in README.
Circle Discord	De-risk	First-line support during build. Tag DevRel on blockers.
Circle-titanoboa SDK	Vyper alignment	Local Vyper simulation of payment logic.
Vyper-agentic-payments	Vyper alignment	Reference implementation for Agent SDK payment flow.
ERC-8004-vyper	Novel + Vyper aligned	Reputation contract deployed on Arc.
AIsa APIs	AIsa alignment + dogfood	Our agents consume AIsa data via Nanopayments in-demo.

8.1 Dogfood Narrative

Both provider and consumer of Nanopayments

Tollgate is the only submission that demonstrates Circle Nanopayments from both sides simultaneously:

- **Provider side.** Our middleware lets publishers accept Nanopayments from any AI scraper.
- **Consumer side.** Our demo agents call AIsa Nanopayment APIs to decide which articles to scrape.

Two layers of Circle Nanopayments in one 90-second demo. Judges see the full-stack thesis play end to end, not just a one-sided payment loop.

8.2 Judge Alignment

Of 22 listed judges, 9 are directly tied to sponsor products: 7 Circle representatives, 1 Vyper DevRel (Luca Franceschini), 1 AIsa CPO (Karen Sheng). Every integration above registers with one of these people. Most teams will use 3 or 4 Circle primitives and skip Vyper and AIsa entirely. We use all 15.

9 Demo Strategy

9.1 The 50+ Transaction Gate

Hard rule from the hackathon brief

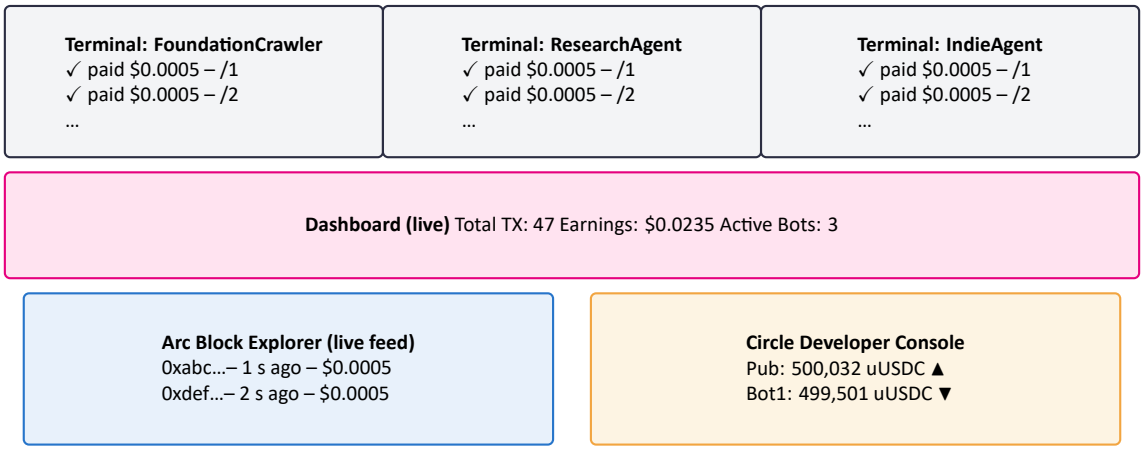
Every submission must demonstrate real per-action pricing at or under $0.01, produce at least 50 onchain transactions in the demo, and include a margin explanation describing why the same model fails with traditional gas costs.

Our target: **60 onchain transactions from 3 different agent wallets in 90 seconds.**

9.2 Three-Agent Network-in-Motion

Agent persona	Wallet	Pace	Pages	TX count
FoundationCrawler	0xF00…	1.5 s / request	30	30
ResearchAgent	0xRE5…	2.0 s / request	20	20
IndieAgent	0x1ND…	3.0 s / request	10	10
Total	3 wallets	Parallel + staggered	60	**60**

9.3 Screen Composition



9.4 Margin Reveal

Metric (60 TX)	Arc	Ethereum	Polygon
Revenue	$0.0300	$0.0300	$0.0300
Gas spent	$0.00120	$30.00	$0.60
Net profit	$0.02880	-$29.97	-$0.57
Margin %	**96%**	**-99,900%**	**-1,900%**

Why Arc wins this math

Arc denominates gas in USDC. Gas cost is fixed at design time, not probabilistic at runtime. Ethereum's native-token gas spikes turn any sub-cent business into a loss on any request. Arc is the only chain where this model is mathematically profitable.

10 Five-Day Build Plan

Day	Focus	Deliverables
1	Foundations	Arc RPC wired, USDC contract referenced, Circle Developer account active, testnet faucet drained into four wallets, first hello-world TX visible on Arc Explorer.
2	Core middleware	Express + Hono + Cloudflare Worker middleware with 402 response shape, x402 integration, HMAC receipt signer, publisher dashboard scaffold.
3	Agent flow	Agent SDK (Node) auto-handles 402, signs via Circle Wallets, retries with proof. AIsa API integration. Circle Gateway multi-chain balance.
4	Reputation + off-ramp	ERC-8004-vyper contract deployed via Circle-titanoboa SDK. Publisher withdraw flow using CCTP. Dashboard live counters via Convex subscriptions.
5	Polish + submission	Record demo through Circle Developer Console and Arc Explorer. Write 1,500-word Product Feedback. Push repo public. Submit.

10.1 Risk Tiering

Tier	What ships
Tier 1 (must ship)	Arc, USDC, Nanopayments, Circle Wallets, x402, Arc Explorer, Circle Console. Without these there is no submission.
Tier 2 (ship if Day 1-3 on track)	AIsa integration, Circle Gateway, ERC-8004-vyper. These turn a viable submission into a winning one.
Tier 3 (document if time runs out)	Bridge Kit full flow, Vyper contract suite, self-hosted facilitator. Slide only if not shippable.

11 Product Inventory

#	Product	MVP?	Who pays	Revenue model
1	Middleware SDK	Yes	Publisher dev	Free, open source
2	Agent SDK	Yes	Agent dev	Free, open source
3	Dashboard	Yes	Publisher	Free tier + Pro $29/mo
4	Hosted Gateway	Year 1	Non-technical pub	5% of earnings
5	Facilitator	Year 1	Infra / enterprise	Per-verify fee
6	Reputation Oracle	Year 1	Agent + publisher	Free read, stake to write
7	Explorer	Year 2	Public	Premium search
8	Analytics API	Year 2	Enterprise publisher	Usage-based
9	WAF Integrations	Year 2	Enterprise	Included Pro+
10	Platform Plugins (WordPress, Ghost, Shopify)	Year 2	Long-tail publisher	Included

11.1 Three-Leg Tripod

What we build in five days

Middleware SDK collects payments. **Agent SDK** sends payments. **Dashboard** proves the flow. That is the tripod. Products 4 through 10 are commercialization and scale, worth sketching on the roadmap slide but not worth building for the demo.

12 Ideal Customer Profiles

12.1 Six Profiles, Two Sides

#	Profile	Side	Priority	Deal size	Adoption speed
P1	Technical content publisher (docs, dev blogs)	Earn	High	Self-serve $0-$199/mo	Days
P2	Trade / niche media with dev resources	Earn	High	Pro $199-$999/mo	Weeks
P3	Mid-tier AI company positioning on ethics	Pay	High	Free SDK + facilitator fee	Days
E1	Enterprise media conglomerate	Earn	Expansion	$5K-$25K/mo	6-12 months
E2	Enterprise internal AI team (Fortune 500)	Pay	Expansion	$50K-$500K/yr	6-9 months
E3	AI research institution (academic)	Pay	Expansion	Free + fee	1-3 months

12.2 Beachhead

Week-1 target after launch

P1: Technical Content Publishers. Docs sites, dev blogs, technical Substacks, engineering community sites. Dev teams install SDKs same-day. Crypto-native, no USDC education needed. Public community distribution (HackerNews, X) is free. Fifty P1 wins become the case-study base that later unlocks P2 trade media and E1 enterprise.

13 Claude Code Build Orchestration

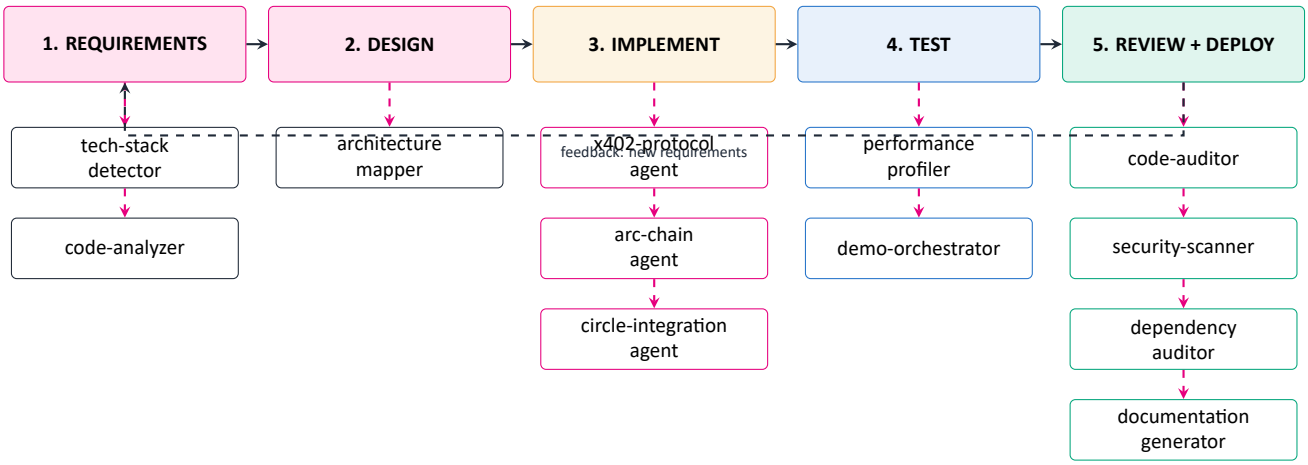
Who codes Tollgate

Tollgate is built by an orchestrated team of Claude Code agents, each specialized for a discrete part of the stack. Twelve agents in total: eight general-purpose agents from the knowledge brain and four Tollgate-specific agents that handle blockchain, Circle, x402, and demo orchestration.
Each agent reads its `AGENT.md` (process + checklist) before running, produces a report in its `REPORT-TEMPLATE.md` format, and hands off to the next agent in the chain. Nothing is improvised. Every phase has an owner.

13.1 Agent Roster

#	Agent	Responsibility in Tollgate build
General-purpose agents (from Claude knowledge brain)		
1	tech-stack-detector	Confirm Node 20, TypeScript, Next.js 16, Convex, Clerk, Cloudflare Workers, Vyper. Flag drift.
2	code-analyzer	Map repo structure at every checkpoint. Produce module-level summary.
3	architecture-mapper	Keep this architecture document in sync with code. Detect diverged diagrams.
4	code-auditor	Review every PR against naming, structure, anti-patterns, correctness, maintainability.
5	security-scanner	Scan 402 flow, HMAC receipts, nonce replay, Circle key storage, Clerk JWT handling, CORS.
6	performance-profiler	Measure hot paths: cached request p95 $<$ 50 ms, cold p95 $<$ 1.2 s, edge KV lookup.
7	dependency-auditor	Audit Circle SDK, Convex SDK, x402 client, ERC-8004 ABI, OpenRouter (if used).
8	documentation-generator	Produce README, SDK docs, install guide, Circle Product Feedback write-up.
Tollgate-specific agents (built for this project)		
9	arc-chain-agent	Arc RPC client, TX signing, contract deployment via Circle-titanoboa, event parsing.
10	circle-integration-agent	Circle Wallets provisioning, Gateway balance queries, CCTP bridge calls, webhook HMAC verify.
11	x402-protocol-agent	402 response builder, facilitator client, nonce generator, receipt HMAC signer and verifier.
12	demo-orchestrator-agent	Fund 3 testnet wallets, launch parallel scrapers, capture TX hashes, produce Arc Explorer bookmarks.

13.2 Orchestration Diagram



13.3 Build Phases Mapped to Agents

Phase	Primary agents	Deliverable
1. Requirements	tech-stack-detector, code-analyzer	Stack confirmation report, repo baseline map.
2. Design	architecture-mapper	This document, kept current.
3. Implement	x402-protocol-agent, arc-chain-agent, circle-integration-agent	Working middleware, agent SDK, dashboard.
4. Test	performance-profiler, demo-orchestrator-agent	p95 benchmarks; 60-TX demo running on testnet.
5. Review	code-auditor, security-scanner, dependency-auditor	PR-by-PR audit reports. No merge without green checks.
6. Deploy	CI / CD pipeline (GitHub Actions)	Vercel preview per PR, Convex deploy on main.
7. Monitor	Sentry, Axiom, Convex dashboard, PostHog	Live alerting; feedback into next requirements cycle.

13.4 Agent Handoff Rules

How agents pass work to each other

- **Read before act.** Every agent reads its `AGENT.md` before starting. Output uses the `REPORT-TEMPLATE.md` format.
- **One source of truth per concern.** Only arc-chain-agent touches Arc RPC. Only circle-integration-agent touches Circle APIs. Only x402-protocol-agent touches 402 and receipt formats. Prevents duplicate or conflicting client code.
- **No merge without audit.** code-auditor, security-scanner, and dependency-auditor must all pass on every PR before merge. No exceptions for demo pressure.
- **Performance gates are code gates.** performance-profiler runs on every PR that touches middleware or facilitator. A regression blocks merge.
- **Demo orchestrator owns the demo.** Day 5 video recording is its job. No code changes accepted on Day 5 except from demo-orchestrator.
- **Feedback loop.** After deploy, monitoring reports feed into new requirement tickets, not ad-hoc bug fixes. Coherence over speed.

13.5 Why This Matters for the Hackathon

Agent orchestration is a judge-signal

Most solo hackers hand-roll everything. Tollgate demonstrates a disciplined, agent-orchestrated build that produces an auditable trail: every PR reviewed by named agents, every integration owned by a specialist, every benchmark enforced automatically. Judges who open the repo see structure, not scaffolding.

This matters because the hackathon brief explicitly rewards production-readiness. The agents are not cosmetic. They enforce the non-functional requirements that this architecture document commits to.

Appendix A: Tech Stack Summary

Layer	Chosen tech
Frontend (dashboard)	Next.js 16 (App Router, React 19), Tailwind 4, Shadcn/ui
Frontend (marketing)	Same repo, static pre-render
Auth	Clerk (@clerk/nextjs), JWT template `convex`
Backend	Convex (queries, mutations, actions, scheduler, search, vector, R2 storage bridge)
Edge runtime	Cloudflare Workers (hosted gateway + telemetry)
Middleware libs	@tollgate/express, @tollgate/hono, @tollgate/cloudflare-worker
Agent libs	@tollgate/agent (Node), tollgate-agent (Python)
Payments	USDC on Arc via Circle Nanopayments + x402
Wallets	Circle Wallets (publisher custody), self-custody optional (agents)
Cross-chain	Circle Gateway (balance), Bridge Kit / CCTP (off-ramp)
Smart contracts	ERC-8004-vyper (reputation), USDC (Circle deployment)
Vyper tooling	Circle-titanoboa-sdk for local simulation
Data upstream	AIsa Nanopayment APIs
Observability	Sentry (errors), Axiom (logs), PostHog (product), Convex dashboard
CI/CD	GitHub Actions → Vercel preview on PR, Convex deploy on merge to main
Domains	tollgate.brianmwai.com (dashboard), demo-news.brianmwai.com (demo site), api.tollgate.brianmwai.com (gateway)

Appendix B: Glossary

Term	Definition
402	HTTP status code `Payment Required`, reserved since RFC 2616 (1997), activated for programmatic use by x402.
Arc	Circle's purpose-built Layer-1 blockchain for stablecoin settlement. USDC is the native gas token.
CCTP	Cross-Chain Transfer Protocol, Circle's native bridging of USDC across supported chains.
Circle Gateway	Unified USDC balance view across chains. Enables single-balance spending.
Circle Wallets	Circle's programmable wallet API, custodial or smart wallet flavors.
ERC-8004	Emerging standard for onchain agent identity, reputation, and validation.
Facilitator	Service that verifies x402 payment proofs against the chain before content is served.
Nanopayments	Circle's primitive for high-frequency, sub-cent transactions with deterministic fees.
Nonce	Single-use identifier in the 402 response that prevents replay attacks.
Receipt	HMAC-signed short-TTL token that lets an agent skip onchain verification on repeat requests.
uUSDC	Micro-USDC, equal to 10^{-6} USDC. Unit in which Tollgate prices are stored.
Vyper	Python-flavored smart-contract language, used for ERC-8004 implementation.
x402	Open web payment standard built on HTTP 402, with facilitator-verified proofs.

Appendix C: References

- Arc Documentation – arc.network/docs
- Circle Developer Docs – developers.circle.com
- Circle Nanopayments – developers.circle.com/nanopayments
- x402 Specification – x402.org
- ERC-8004 Draft – EIP-8004

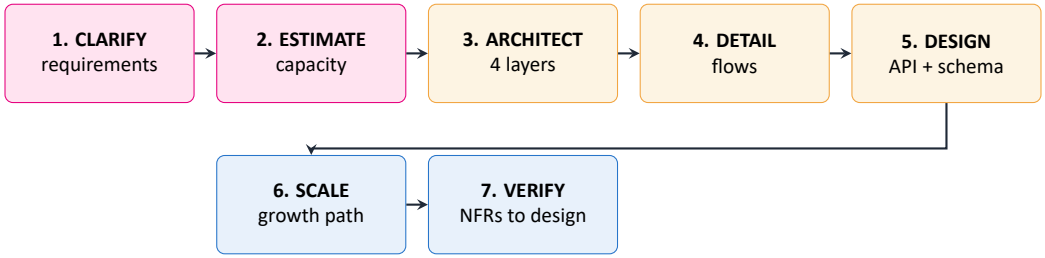
- Circle Github – github.com/circlefin
- AIsa API Collection – github.com/aisa-xyz
- Tollgate Repository – github.com/brn-mwai/tollgate

Appendix D: Reference Diagrams from Knowledge Brain

Provenance

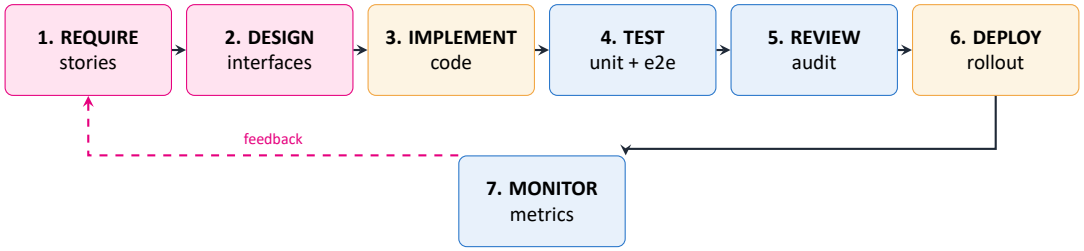
The following diagrams come from the engineering knowledge base at `Claude_brain/diagrams/`. Each is a general-purpose pattern applied here to Tollgate. Keeping them in this document means the build team can see both the abstract pattern and the Tollgate instantiation side by side.

D.1 Seven-Step System Design Framework



Applied: Sections 1 through 7 of this document.

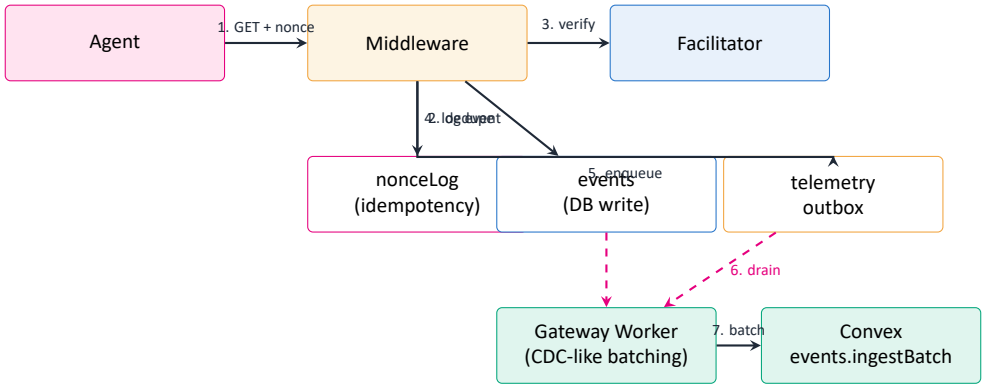
D.2 Coding System Seven-Phase Process



Applied: Section 13 Claude Code Build Orchestration.

D.3 Payment Flow with Idempotency and Outbox

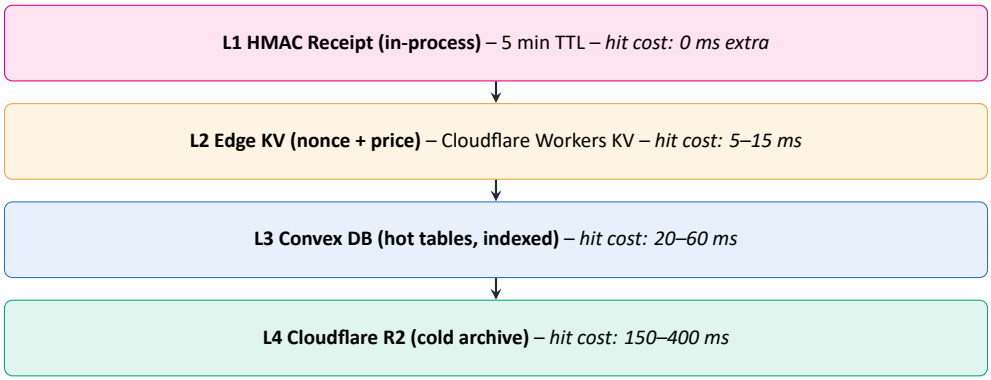
Tollgate reuses the Stripe-style payment pattern: idempotency keys prevent double-charge, and a transactional outbox prevents event-DB divergence.



Applied: nonceLog table plus edge telemetry outbox plus Convex batching in Section 5.

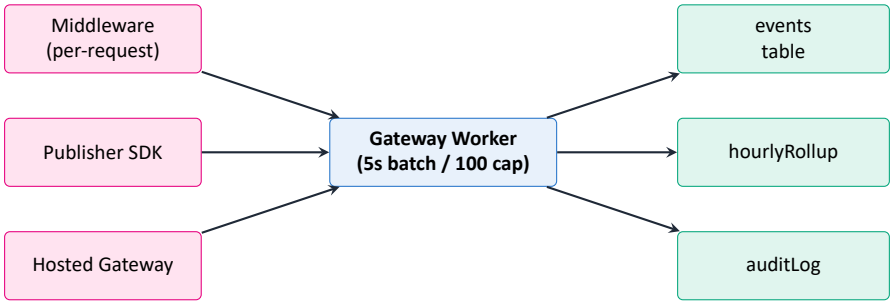
D.4 Caching Strategy Layers

Each layer sits closer to the agent than the next. A cache hit at any layer skips everything below.



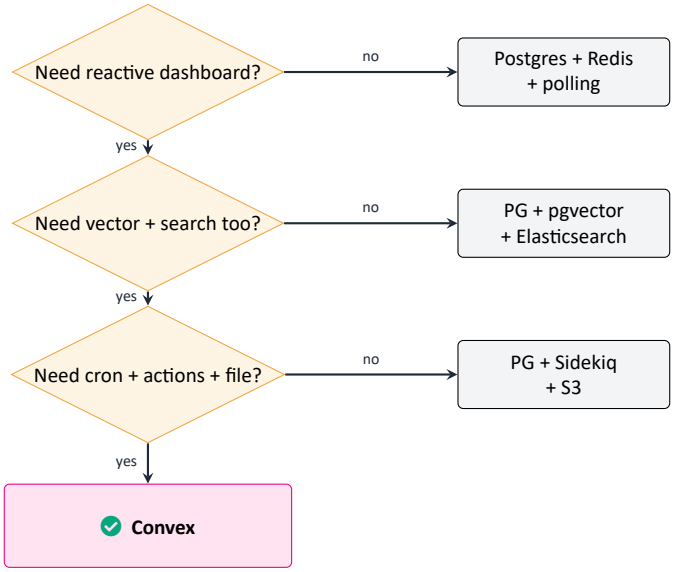
Applied: the receipt collapses 50 requests into 1 onchain TX. Cache invalidation is TTL-only, matching the brain guidance that short TTLs beat complex invalidation.

D.5 Message Queue Pattern (Telemetry Outbox)



Applied: fanout pattern plus outbox means new consumers (audit, analytics, compliance export) can subscribe without touching publisher code.

D.6 Database Decision That Led to Convex



Applied: Tollgate hits yes on all three gates (reactive dashboard, vector + search for agent clustering, cron plus actions plus file storage). Convex replaces four systems with one.

Tollgate turns every AI scraper request from a theft into a transaction.

Built for the Agentic Economy on Arc, Circle Hackathon 2026.